



Lab Number:	09
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	Subqueries
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab



Table of Contents

1 What Is a Subquery?	1
1.1 Characteristics	1
1.2 General Syntax	1
1.3 Examples	1
2 Types of Subqueries by Return Value	3
2.1 Scalar	3
2.2 List / Column	3
2.3 Table / Rowset / Derived Table / Virtual Table	4
2.4 Empty Set	4
3 Types of Subqueries by Correlation	4
3.1 Non-Correlated Subquery	4
3.2 Correlated Subquery	4
4 Special-Purpose Operators	5
4.1 IN / NOT IN	5
4.2 EXISTS / NOT EXISTS	5
4.3 ANY / SOME	6
4.4 ALL	6
5 Types of Subqueries by Position	6
5.1 SELECT Clause	6
5.2 FROM Clause	7
5.3 WHERE Clause	7
6 DML Subqueries	7
6.1 INSERT using Subquery	8
6.2 UPDATE using Subquery	8
6.3 DELETE using Subquery	8



1 What Is a Subquery?

A subquery is a SQL query (**SELECT** statement) nested inside another query/statement. Subqueries provide the ability to **use the result of one query as input to another**, enabling more flexible and powerful data retrieval.

1.1 Characteristics

- A subquery is also called an **inner query** whereas the main query containing the subquery is called an **outer query**.
- Subqueries can be used in DQL (**SELECT**) or DML (**INSERT**, **UPDATE**, and **DELETE**) statements.
- A subquery is almost always **enclosed within parentheses**
- Subqueries are usually executed prior to the containing/main/outer statement.
- The output of a subquery is used as an input for the outer query.

1.2 General Syntax

```
-- Main query
SELECT column(s)
FROM table(s)
WHERE [column] operator (
    -- Subquery
    SELECT column(s) FROM table ...
);
```

1.3 Examples

- **Comparing Against an Aggregate:**

```
-- Show employees who earn more than the average salary
SELECT name, salary
FROM employees
WHERE salary > (
```



```
SELECT AVG(salary) FROM employees  
);
```

- **Finding Top / Extreme Values:**

```
-- Show employees with the maximum salary  
SELECT name, salary AS highest_salary  
FROM employees  
WHERE salary = (  
    SELECT MAX(salary) FROM employees  
);
```

- **Finding Records Based on Another Table:**

```
-- List students from CS and IT departments  
SELECT name, department_id  
FROM students  
WHERE department_id IN (  
    SELECT id FROM departments WHERE name = 'CS' OR name = 'IT'  
);
```

- **Finding Existence of Records:**

```
-- Show departments without any students  
SELECT name  
FROM employees  
WHERE department_id NOT IN (  
    SELECT id FROM departments  
);  
  
-- OR  
  
SELECT name  
FROM departments d  
WHERE NOT EXISTS (  
    SELECT 1 FROM students s WHERE s.department_id = d.id  
);
```

- **Row-by-Row Comparison:**



```
-- Find students with highest CGPA per department
SELECT name, department_id, cgpa
FROM students s1
WHERE cgpa = (
    SELECT MAX(cgpa) FROM students s2 WHERE s2.department_id =
s1.department_id
);
```

2 Types of Subqueries by Return Value

A subquery may return different types of result sets. This determines how a particular subquery may be used and which operators the containing statement may use to interact with the data the subquery returns.

Following are the four types of result sets returned by a subquery:

2.1 Scalar

A scalar subquery returns a single value (1 row, 1 column). Such subqueries can be used anywhere a single value is expected, as in the right side of a comparison expression.

Example:

```
(SELECT MAX(salary) FROM employees)
```

2.2 List / Column

This type of subquery returns multiple values (1 column, multiple rows). Such subqueries can be used anywhere a list of values is expected, such as when using the **IN** operator.

Example:

```
(SELECT department_id FROM departments)
```



2.3 Table / Rowset / Derived Table / Virtual Table

This subquery returns multiple rows and columns, representing a virtual table. Such subqueries can be used anywhere a table is expected, such as when using the **FROM** clause.

Example:

```
(SELECT id, name FROM students WHERE cgpa > 3.0)
```

2.4 Empty Set

Some subqueries may return no rows and instead return a **NULL** value. Such subqueries can be used to check the presence/absence of records, such as when using the **EXISTS** operator.

Example:

```
(SELECT id FROM students WHERE cgpa > 5.0)
```

3 Types of Subqueries by Correlation

3.1 Non-Correlated Subquery

- Independent of outer query
- Can run standalone
- Result passed to outer query

Example:

```
SELECT name, salary  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary) FROM employees  
);
```

3.2 Correlated Subquery

- Depends on outer query



- Runs once per row of the outer query
- Typically used for row-wise comparisons

Example:

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e1.department_id
);
```

4 Special-Purpose Operators

4.1 IN / NOT IN

Checks if a value matches any value in the subquery result.

Example:

```
SELECT name
FROM students
WHERE department_id IN (
    SELECT id FROM departments WHERE name='CS'
);
```

4.2 EXISTS / NOT EXISTS

Checks if the subquery returns any rows.

Example:

```
SELECT d.name
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM students s WHERE s.department_id = d.id
);
```



```
);
```

4.3 ANY / SOME

Compares value to any value in the subquery.

Example:

```
-- Employees earning more than ANY employee in department 1
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary FROM employees WHERE department_id = 1
);
```

4.4 ALL

Compares value to all values in subquery.

Example:

```
-- Employees earning more than ALL employees in department 1
SELECT name, salary
FROM employees
WHERE salary > ALL (
    SELECT salary FROM employees WHERE department_id = 1
);
```

5 Types of Subqueries by Position

5.1 SELECT Clause

Subquery returns a value to include in result columns.

Example:

```
SELECT
```




```
name,  
  (SELECT name FROM departments WHERE id = department_id) AS dept_name  
FROM students;  
  
SELECT  
  name,  
  (SELECT AVG(budget) FROM departments) AS average_budget,  
  budget - (SELECT AVG(budget) FROM departments) AS difference  
FROM departments;
```

5.2 FROM Clause

Subquery used as a derived table.

Example:

```
SELECT t.department_id, AVG(t.salary)  
FROM (SELECT * FROM employees) t  
GROUP BY t.department_id;
```

5.3 WHERE Clause

Subquery used for filtering.

Example:

```
SELECT name  
FROM students  
WHERE cgpa > (  
  SELECT AVG(cgpa) FROM students  
);
```

6 DML Subqueries

Subqueries can be used with **INSERT**, **UPDATE**, **DELETE** statements for more dynamic operations.



6.1 INSERT using Subquery

Example:

```
INSERT INTO high_achievers (name, cgpa)
SELECT name, cgpa
FROM students
WHERE cgpa >= 3.5;

-- OR

INSERT INTO high_achievers (name, cgpa)
SELECT name, cgpa
FROM students
WHERE cgpa > (SELECT AVG(cgpa) FROM students);
```

6.2 UPDATE using Subquery

Example:

```
UPDATE employees
SET bonus = 1000
WHERE salary < (
    SELECT AVG(salary) FROM employees
);
```

6.3 DELETE using Subquery

Example:

```
DELETE FROM employees
WHERE department_id IN (
    SELECT id FROM departments WHERE name='IT'
);
```